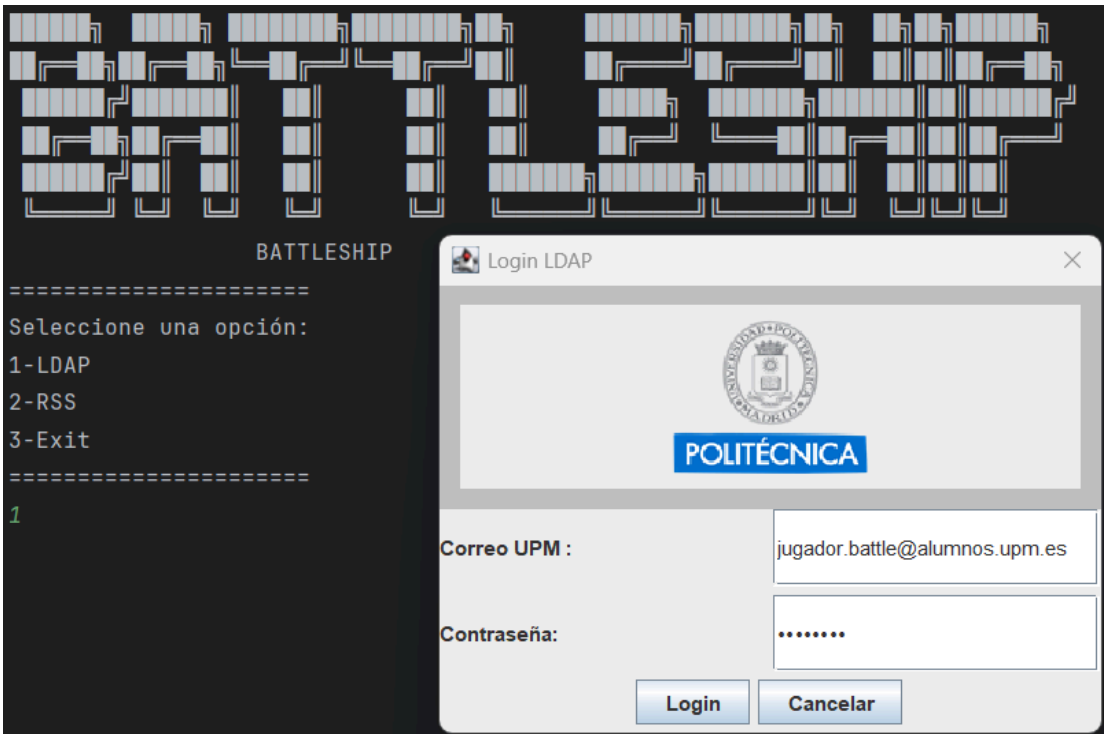
PRUEBAS TRAZABILIDAD

1.0 Para iniciar sesión , ejecutaremos el programa:

```
public void start() {
   menu.addOptions(new LDAP());
   menu.addOptions(new RSS());
   menu.addOptions(new ExitVista());
   menu.run();
}
```



Deberá de cumplir los siguientes requisitos:

Feature #18977: Gestión de usuarios
Requisito_Funcional #18987: Registro de jugadores
« Anterior | 13/28 |

Requisitos de la contraseña

Añadido por Sergio Sanz Nieto hace 3 meses. Actualizado hace alrededor de 2 meses.

Estado:	Especificando	**Fecha de inicio:**	2025-03-11
Prioridad:	Alta	**Fecha fin:**	
Asignado a:	Sergio Sanz Nieto		

Descripción

Especificación del formato de la contraseña

- Entrada: introducir una contraseña,
- Proceso: comprobar que contenga mayúsculas, minúsculas, números y símbolos con tamaño mínimo de 6 caracteres y 12 de máximo
- Salida: mensaje al usuario validando o no la contraseña introducida y almacenar la contraseña (cifrada, LOPD-GDD)
- Comentario: Se deberá cumplir correctamente lo que especifica con la Ley Orgánica 3/2018 de Protección de Datos Personales y Garantía de los Derechos Digitales (LOPD-GDD).

Feature #18977: Gestión de usuarios
Requisito_Funcional #18987: Registro de jugadores

Validación de nombres

Añadido por Sergio Sanz Nieto hace 3 meses. Actualizado hace alrededor de 2 meses.

Estado:	Especificando	**Fecha de inicio:**	2025-02-21
Prioridad:	Alta	**Fecha fin:**	
Asignado a:	Sergio Sanz Nieto		

Descripción

Se validará que el usuario introducido cumpla con ciertas condiciones y que no esté previamente en una lista negra

- Entrada: nombre del usuario introducido por un usuario
- Gestión: se comprobará que el nombre del usuario contenga entre 3 y 10 caracteres de solo letras y numeros que no esté contenida en la lista negra
- Salida: mensaje al usuario de si su usuario es valido o no

Feature #18977: Gestión de usuarios
Requisito_Funcional #18987: Registro de jugadores

Requisitos de email

Añadido por Yijun Jiang Chen hace 3 meses. Actualizado hace alrededor de 2 meses.

Estado:	Especificando
Prioridad:	Alta
Asignado a:	Sergio Sanz Nieto

Descripción

El email será correo limitado a user UPM usar LDAP de UPM .

Entrada: Correo electrónico (debe ser un correo UPM reconocido por LDAP) tipo @*****.upm.es .
Gestión: Se validará el correo electrónico utilizando el servicio de LDAP de UPM.
Salida: Si el correo es válido y reconocido por LDAP, se crea el usuario en la base de datos.

```
public void login(String credencial) {
    if (credencial != null) {
        Jugador jugador = cJugador.getJugador(credencial);
        if (jugador == null) {
            System.out.print("Nombre de usuario: ");
            String nombre = scanner.nextLine();
            while (!cJugador.nombreUsable(nombre)) {
                System.out.println("No se puede usar este nombre introduce de
nuevo: ");
                nombre = scanner.nextLine();
            }
            this.jugadoract = new JugadorHumano(nombre, credencial);
            cJugador.crearJugador(jugadoract);
        } else {
            this.jugadoract = jugador;
        }
        logged();
    }
}
```

```
public boolean nombreUsable(String username) {
    if (username == null || username.trim().isEmpty()) {
        System.out.println("El nombre de usuario no puede estar vacío.");
```

```
        return false;
    }
    username = username.trim();

    if (username.length() < 3 || username.length() > 10 ||
!username.matches("[a-zA-Z0-9]+")) {
        System.out.println("Nombre de usuario para el juego no válido. Debe
tener entre 3-10 caracteres alfanuméricos.");
        return false;
    }
    if (LISTA_NEGRA.contains(username.toLowerCase())) {
        System.out.println("El nombre '" + username + "' no está permitido
(lista negra).");
        return false;
    }
    if (getJugadorPorNombreJuego(username) != null) {
        System.out.println("El nombre de juego '" + username + "' ya está en
uso.");
        return false;
    }
    return true;
}
```

Compo se pueden observar en las siguientes imagenes cumplen los requisitos

```
1
Nombre de usuario: al
Nombre de usuario para el juego no válido. Debe tener entre 3-10 caracteres alfanuméricos.
No se puede usar este nombre introduce de nuevo:
sdagjsdgnaslkjdn234
Nombre de usuario para el juego no válido. Debe tener entre 3-10 caracteres alfanuméricos.
No se puede usar este nombre introduce de nuevo:
ask&//8.#
Nombre de usuario para el juego no válido. Debe tener entre 3-10 caracteres alfanuméricos.
No se puede usar este nombre introduce de nuevo:
```

```
No se puede usar este nombre introduce de nuevo:
josito12
Jugador 'josito12' registrado.
Bienvenido josito12
Seleccione una opción:
1-Jugar Partida
2-Ver top 10 puntuacion
3-Dar de baja
4-Logout
5-Exit
======================
```

2.0 Para dar de baja a un jugador, se deberá eliminar de la base de datos

Feature #18977: Gestión de usuarios

« Anterior | 28/28 | Siguiente »

Baja de Jugador

Añadido por Yijun Jiang Chen hace 3 meses. Actualizado hace 2 meses.

Estado:	Especificando	**Fecha de inicio:**	2025-02-21
Prioridad:	Media	**Fecha fin:**	
Asignado a:	Sergio Sanz Nieto		

Descripción

Citar

Permite a los usuarios darse de baja del sistema, eliminando sus datos personales y estadísticas asociadas.

- Entradas: Solicitud de baja: Realizada por el usuario (por ejemplo, mediante un botón o formulario).
- Procesamiento: Verificar la identidad del usuario solicitante. Proceder a la eliminación de los datos personales y estadísticas del usuario en la base de datos.
- Salidas: Confirmación de la baja exitosa. Mensaje de error si ocurre algún fallo durante el proceso de eliminación.
- Consideraciones Adicionales: Confirmar con el usuario la acción de baja para evitar eliminaciones accidentales y gestionar adecuadamente la persistencia de datos históricos si es requerido por normativas.

```
public void eliminar(Jugador jugador) {
    if (jugador == null) return;
    if (jugador.isAdmin()) {
        System.out.println("Acción no permitida: Los
administradores no pueden ser eliminados.");
        return;
    }

    boolean removed = this.jugadores.remove(jugador);
    if (removed) {
        this.persistencia.guardarTodosLosJugadores(this.jugadores);

this.persistencia.eliminarArchivoPuntuaciones(jugador.getNombre())
;
        System.out.println("Jugador '" + jugador.getNombre() + "'
eliminado.");
    } else System.out.println("Jugador '" + jugador.getNombre() +
"' no encontrado en la lista para eliminar.");
}
```

```
Bienvenido josito12
Seleccione una opción:
1-Jugar Partida
2-Ver top 10 puntuacion
3-Dar de baja
4-Logout
5-Exit
======================
3
Jugador 'josito12' eliminado.
```

Como se puede observar ‘josito12’ ya no está en la base de datos

```
adminFIS 085427b18b067e5c9081c8e679a9070a ADMIN
normal1 d80a8428e5b3cc16be2b0d5799d5b7a0
xshan 559bda931f4ad34582b774504535 84b8
normal2 10def4a53103bf9d09d53b53b02ad5bd
qwe 84b761ea0af3bbf54294df72b321d85d
DonJuan bdf6f39b92f858cef65514e664555e09 ADMIN
Juan ed745e64aa689ed205323165d424cc7e
DANIEl 72f9130be0575b9365d76dbe4eac335d
jose 24702c443c1f73853ff534b9640bc8b8
```

Para poder comprobarlo, se han creado unas pruebas

```
package CITIM21_09;


import junit.framework.TestCase;
import CITIM21_09.controller.CJugador;
import CITIM21_09.model.Jugador;
import CITIM21_09.model.JugadorHumano;


import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
```

```
import java.util.List;


public class CJugadorTest extends TestCase {


   private CJugador controlador;


   protected void setUp() throws Exception {
       super.setUp();
       controlador = new CJugador();


       List<Jugador> jugadores = new ArrayList<>();
       jugadores.add(new JugadorHumano("pepe69", "cred1")); //
nombre ya en uso
       jugadores.add(new JugadorHumano("calendar", "cred2"));
       controlador.setJugadores(jugadores);
   }


   public void testCP1_V1V2V3() {
       assertTrue(controlador.nombreUsable("jos"));
       System.out.println("jos"+" aceptado");
   }


   public void testCP2_V1V2N4() {
       assertFalse(controlador.nombreUsable("pepe69")); // en uso
       System.out.println("pepe69"+" nombre ya existe");


   }


   public void testCP3_V1N3V3() {
       assertFalse(controlador.nombreUsable("?%”%/&(__.")); //
caracteres inválidos
       System.out.println("?%<UNK>%/&(__. "+" caracteres
invalidos");


   }


   public void testCP4_N1V2V3() {
```

```
        assertFalse(controlador.nombreUsable("A")); // longitud
inválida
        System.out.println("A"+" longitud infeior a 3");
    }


    public void testCP5_N2V2V3() {
        assertFalse(controlador.nombreUsable("fuhdfjwkadfop")); //
longitud inválida
        System.out.println("fuhdfjwkadfop"+" longitud superior a
10");

    }

    public void testEliminarJugador() {
        assertFalse(controlador.nombreUsable("pepe69"));

        Jugador jugador =
controlador.getJugadorPorNombreJuego("pepe69");
        assertNotNull("Jugador no encontrado antes de eliminar",
jugador);

        controlador.eliminar(jugador);

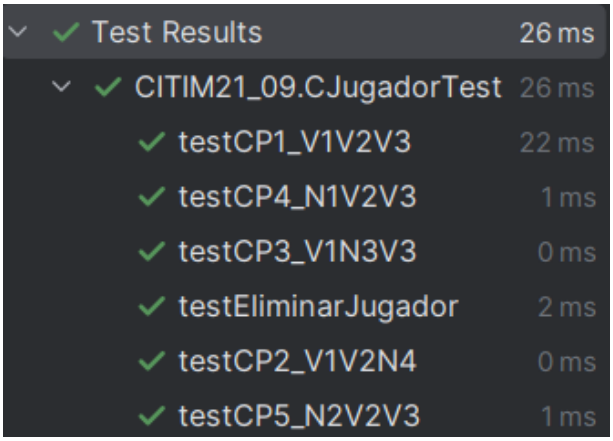
        assertTrue("El nombre debería estar disponible tras
eliminar", controlador.nombreUsable("pepe69"));

        boolean sigueEnArchivo = false;
        try (BufferedReader reader = new BufferedReader(new
FileReader("src/main/resources/Users/datos.txt"))) {
            String linea;
            while ((linea = reader.readLine()) != null) {
                if (linea.toLowerCase().startsWith("pepe69;")) {
                    sigueEnArchivo = true;
                    break;
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
            fail("Error al leer el archivo de jugadores");
        }

        assertFalse("Jugador aún presente en el archivo",
sigueEnArchivo);

        System.out.println("Test de eliminación completado
correctamente.");
    }
```

```
}
```


Test Results 26 ms
CITIM21_09.CJugadorTest 26 ms
testCP1_V1V2V3 22 ms
testCP4_N1V2V3 1 ms
testCP3_V1N3V3 0 ms
testEliminarJugador 2 ms
testCP2_V1V2N4 0 ms
testCP5_N2V2V3 1 ms